

01

Как работает LP

Механизм покупки, продажи и физических ограничений

$$\max \sum_t [\text{price}_t * (\text{discharge}_t - \text{charge}_t) * dt - k_{\text{deg}} * (\text{charge}_t + \text{discharge}_t) * dt]$$

LP смотрит на 24 часа цен. Если цена низкая - выгодно зарядить батарею. Если цена высокая - выгодно разрядить и продать энергию. Он выбирает не одно действие, а весь безопасный профиль charge/discharge/SOC.

- SOC: 5-95% емкости
- charge/discharge $\leq$ max power
- $\text{SOC}_{t+1} = \text{SOC}_t + \text{charge} * \sqrt{\text{RTE}} * dt - \text{discharge} / \sqrt{\text{RTE}} * dt$
- Коммитится только первый час rolling horizon

Результат: BUY если $\text{net_power} < 0$, SELL если > 0 , HOLD если около 0

market_execution_enabled=false





02

Strict Similar Benchmark

Как считается regret и связь с oracle

$$\text{regret} = \text{oracle_value} - \text{achieved_value}$$

Strict benchmark не обучается. Он берет похожий прошлый день как прогноз: для Tue-Fri чаще yesterday, иначе same weekday last week. Потом LP строит schedule по этому прогнозу.

- achieved_value: schedule из strict forecast, оцененный на actual prices
- oracle_value: тот же LP, но с actual future prices
- oracle - это верхняя планка, не реальный доступный режим
- Меньше regret = лучше decision quality

Benchmark нужен как честный frozen comparator для V2+, DT и forecast models.

market_execution_enabled=false



	—	—	—	—
—	—	—	—	—
—	—	—	—	—
—	—	—	—	—



03

Raw Data

Какие данные входят сырыми и какой формы

hourly row = timestamp + tenant_id/source + value columns + provenance

Основной raw input - hourly rows: timestamp + price/weather/source metadata. Украинская цена DAM одна для market zone IPS; tenants отличаются погодой, координатами, BESS параметрами, SOC и economics.

- OREE DAM: hourly price, UAH/MWh, market zone IPS
- Open-Meteo: hourly weather by tenant coordinates/timezone
- Tenant config: capacity, power, SOC limits, degradation cost
- Synthetic fallback rows are demo stability only, not measured-performance claims

Pipeline shape: Bronze observed rows -> Silver features -> Gold schedule/evidence rows.

market_execution_enabled=false



04

Features

Что создается, как выбирается и где используется

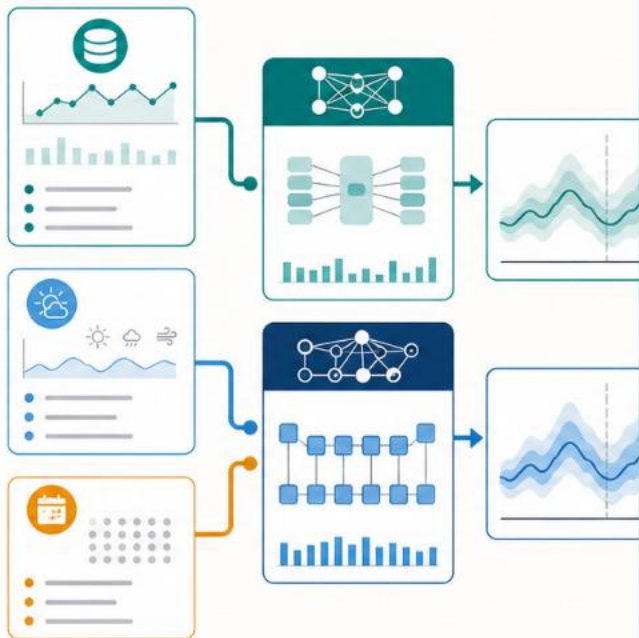
```
features_t = calendar + lags + rolling stats + weather + BESS + candidate context
```

Features создаются после source/governance checks. Они описывают время, историю цен, погоду, батарею и candidate schedule. Важное правило: нельзя использовать future actual/oracle/regret как input.

- Forecast stage: calendar, lags, rolling price stats, forecast-available weather
- LP stage: forecast vector + BESS limits + degradation cost
- V2+/DT stage: prior context, candidate family, dispatch/SOC/throughput summaries
- Leakage guard blocks actual/oracle/regret/labels from selector inputs

Features are chosen by availability at decision time, not by after-the-fact usefulness.

```
market_execution_enabled=false
```



05

TFT и NBEATSx

Их роль в V2+ и DT strategy

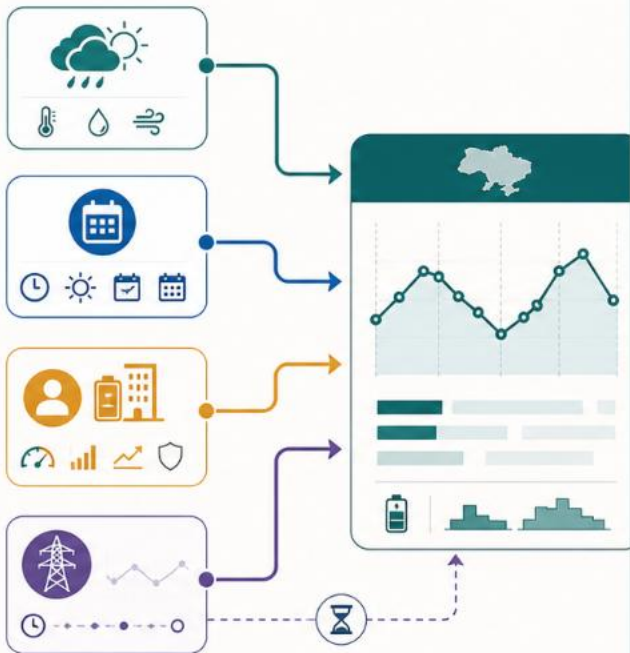
history + exogenous features -> forecast vector -> LP/candidates -> evidence

TFT/NBEATSx - это forecast adapters. Они дают hourly price forecast and explanatory signals. Дальше LP/candidate scorer превращает прогноз в feasible schedules; V2+/DT выбирают между candidate schedules.

- Они не являются controller и не отправляют market bids
- TFT полезен еще и VSN/explainability evidence
- NBEATSx/TFT сравниваются через same frozen LP/oracle scorer
- Текущие claims: adapter readiness/research evidence, not SOTA execution

Short version: forecast layer, not execution layer.

market_execution_enabled=false



06

Exogenous Features

Что это, какие используются, что с Poland

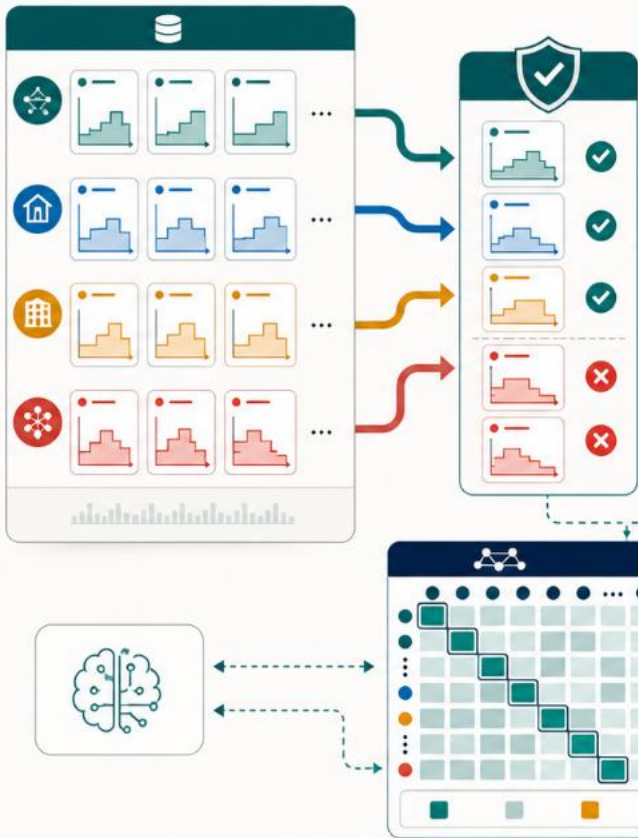
target = UA DAM; exogenous = context, not replacement target

Exogenous features - внешние сигналы, которые известны до решения и помогают объяснить price regime: weather, calendar, neighboring-market context, tenant context.

- Used/available: weather, calendar, tenant/BESS context, lagged market features
- Poland lane: ENTSO-E PL day-ahead price aligned as lag-24 context
- Poland features: lag-24 level, deltas, daily spread/rank, peak/trough hour
- Governance blocks EU/PL rows from becoming Ukrainian training targets

Current Poland result is not promoted: mechanically usable, but weaker than frozen V2+.

market_execution_enabled=false



07

Schedule Selection

Как V2+ создает много schedules и как DT выбирает safe schedules

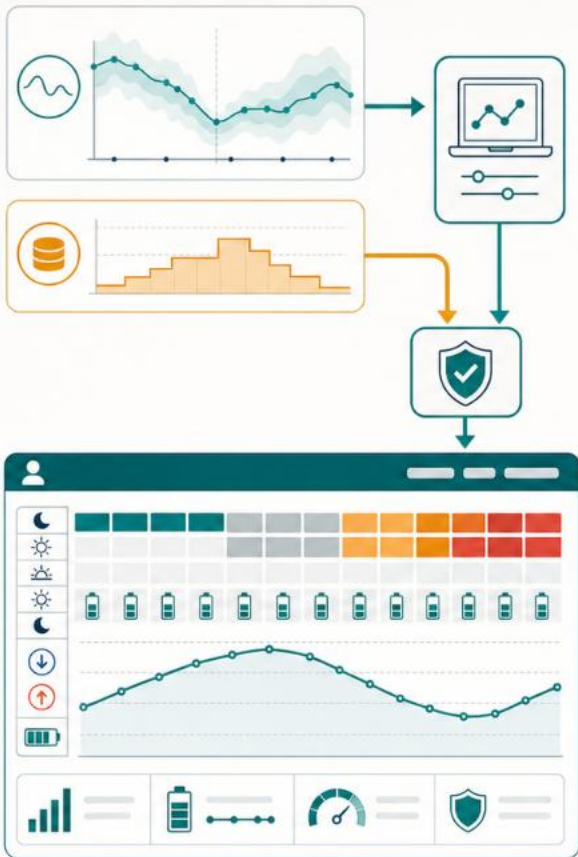
candidate library -> safety/feasibility -> prior-only selector -> selected family

V2+ builds a candidate library: several schedule families perturb or guard the baseline schedules, then each candidate is LP/oracle-scored offline. The selector chooses the lowest prior-regret/value candidate, with fallback to frozen V2.

- V2+ families include extrema perturbation, spread penalty, neighborhood shift, block reconciliation, terminal SOC target
- DT-shadow does not output raw hourly actions; it predicts when switching away from V2+ is safe
- It switches only if predicted improvement clears threshold and tail-risk guard passes
- Otherwise it abstains and keeps V2+

All of this is offline recommendation evidence, not market execution.

market_execution_enabled=false



08

From Prediction to Recommendation

How forecast/candidate becomes feasible LP schedule

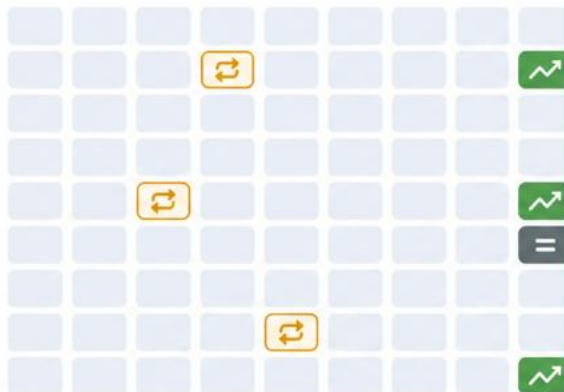
stored dispatch vector = [dispatch_mw at hour 1...24] plus SOC/action metadata

If the selected input is a forecast, LP solves charge/discharge/SOC under battery limits and creates a feasible 24h schedule. If the selected input is an existing candidate, the system reuses its stored dispatch vector.

- forecast vector $\rightarrow$ LP $\rightarrow$ dispatch_mw_vector + soc_fraction_vector
- candidate row already stores dispatch_mw_vector from earlier LP/candidate scoring
- operator preview reads committed_action and committed_power_mw
- Gatekeeper blocks candidates outside physical/market envelope

The output is **BUY/SELL/HOLD recommendation schedule**, not ProposedBid.

market_execution_enabled=false



09

Current DT Shadow Candidate

Why 3 wins and 1 tie; how it got about 3.8% better than V2+

mean regret: V2+ 174.77 UAH -> selected 168.16 UAH = 3.78% lower

The selector was conservative. Out of 90 final anchors, it kept V2+ for 86 and switched only 4 times to strict_reference. Those 4 switches produced 3 lower-regret wins and 1 equal-regret tie, with 0 losses.

- Wins: Kyiv mall, Lviv office, Odesa hotel at 2026-04-15T23:00
- Tie: Kharkiv hospital at the same anchor
- It noticed a prior-context/candidate-family pattern where strict_reference looked safer than V2+
- It did not see future oracle; oracle/regret scored the choice afterward

Boundary: V2+ remains default; DT/LAVA not promoted;
market_execution_enabled=false.

market_execution_enabled=false